

Pong Project Summary

By Idan Nudel

Introduction

Pong is a classic arcade game from 1972, one of the first video games, where two players or a player against the computer play virtual tennis. In this project, I developed a version of the Pong game in the CVI environment using the C programming language, featuring an interactive graphical interface, support for player vs. computer or player vs. player modes, different difficulty levels, and real-time score management.

Objectives

The goal of the project was to create a functional Pong game with the following elements:

- Paddle control using arrow keys (for Player 1) and W/S keys (for Player 2 in multiplayer mode).
- Ball movement with realistic physics, including variable bounce angles based on the ball's impact position relative to the paddle.
- Support for playing against the computer with an AI model offering different difficulty levels (Easy, Medium, Hard, Impossible).
- A graphical interface mimicking the original game with a score display.
- Game state management, including pause, restart, and return to the main menu.
- A user interface with a menu for selecting game mode and difficulty level.

Game Architecture

The game is divided into several main modules:

1. Graphical Interface: Managed using CVI.
2. Game Logic: Handled partly in the main file and partly in a DLL, including ball movement, paddle movement, win detection, and bounce angle calculations.
3. Graphics: Manages the display, image loading, and use of graphical objects.

Software Features

- Support for two game modes: vs. computer (AI) or vs. another player.
- Different difficulty levels (Easy, Medium, Hard, Impossible) affecting the computer's response speed, with higher difficulties increasing the AI's reaction speed.

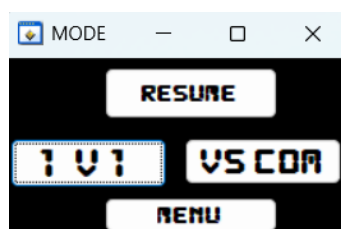
- Panel-based user interface, including a main menu, game mode selection, difficulty selection, and game screen.
- Support for Pong logic: paddle movement, ball bouncing, score calculation, and game end when a player reaches 5 points.
- Use of Canvas for drawing paddles, the ball, and the center line.
- A timer for periodic updates of ball movement and the AI model.
- Keyboard input management for detecting paddle movement.
- Use of a DLL to manage core game logic, including ball movement, AI movement, and bounce angle calculations.

Usage Instructions

- On the main screen, click “Start” to proceed to game mode selection.



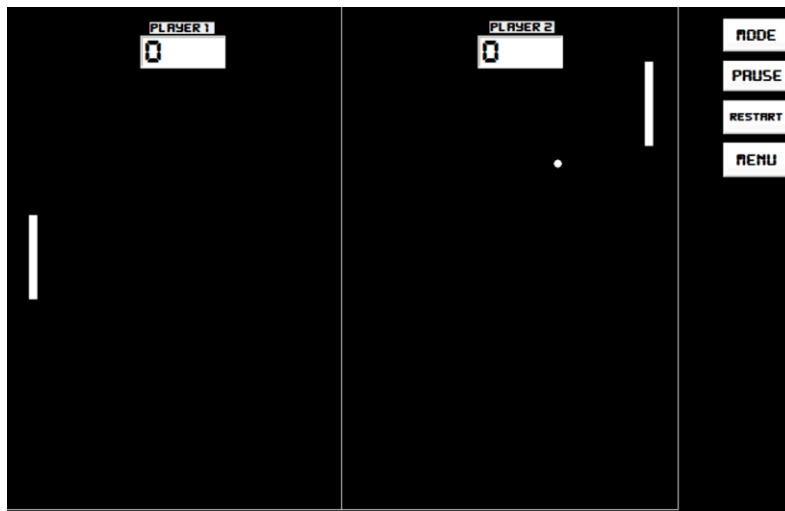
- Choose a game mode: vs. computer (Vs AI) or vs. another player (Vs Player).



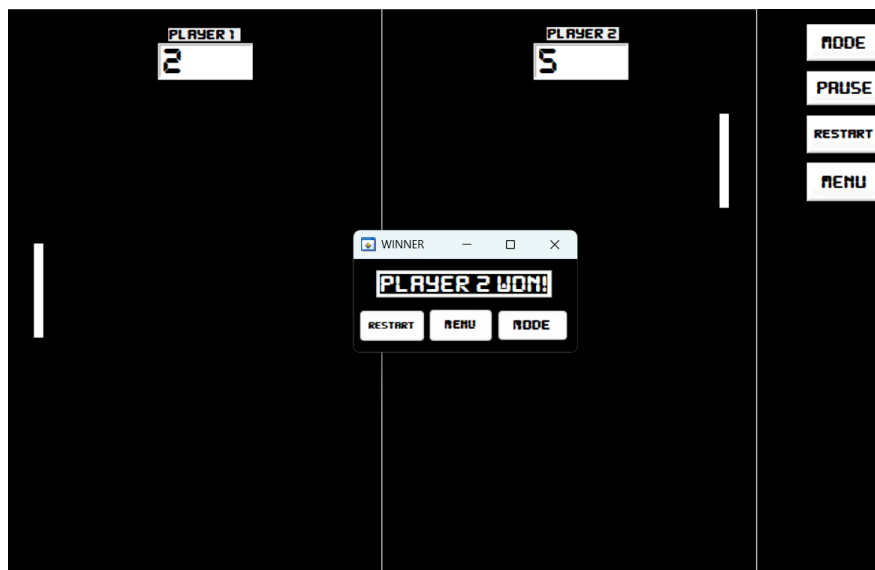
- If playing against the computer, select a difficulty level (Easy, Medium, Hard, Impossible).



- In the game, use the arrow keys (Up/Down) to control the left paddle and W/S keys to control the right paddle (in two-player mode).



- The goal is to score 5 points before the opponent by hitting the ball past their paddle.
- Scoring: One point is awarded when the ball passes the opponent's paddle.
- When a player reaches 5 points, the game stops, and a window displays the winner, offering options to restart, select a new game mode, or return to the main menu.



Graphical Panels

- Start Screen: Includes "Start" and "Quit" buttons.
- Mode Selection Screen: Allows choosing between vs. computer or vs. another player, with an option to return to an active game.
- Difficulty Selection Screen: Allows choosing difficulty levels for vs. computer mode.
- Game Screen: Includes a Canvas for gameplay, score display, and control buttons (Pause, Menu, Restart).

- **Victory Screen:** Displayed when a player reaches 5 points, showing the winner's name and offering options to play again, return to the menu, or select a new game mode.

Main Functions

- **DrawGame:** Draws the paddles, ball, center line, and game frame on the Canvas.
- **MoveBall:** Updates the ball's position based on its speed, handling bounces and scoring (implemented in the DLL).
- **MoveAI:** Controls the computer paddle's movement based on the ball's position and difficulty level (implemented in the DLL).
- **BallBounceAngle:** Calculates the ball's bounce angle after hitting a paddle, including jitter for realism (implemented in the DLL).
- **ResetBall:** Resets the ball's position to the center of the screen (implemented in the DLL).
- **UpdateScoreDisplay:** Updates the score display for both players on the screen (via Numeric).
- **HandleWin:** Manages game termination when a player reaches 5 points, displaying a message and allowing a restart.
- **RestartGame:** Resets the game state, including scores, paddle positions, and ball position.
- **TimerCallback:** Updates the game state on each timer tick, including ball and AI movement.
- **GamePanelEventHandler:** Handles keyboard input for paddle movement control.
- **StartGameFromMenu:** Activates the game mode selection menu from the start menu.
- **PauseGame:** Pauses or resumes the game and updates the user interface accordingly.
- **BackToMainMenu:** Returns the user to the main menu and resets the game state.
- **ModeVsAI:** Activates vs. computer mode and proceeds to difficulty selection.
- **ModeVsPlayer2:** Activates vs. player mode and starts the game.
- **SetDifficultyEasy/Med/Hard/Imm:** Sets the difficulty level and starts the game.
- **ResumeGame:** Resumes a paused game by restarting the timer.

- MyQuitStartPanel: Closes the program from the start menu.

Challenges and Solutions

During development, I encountered several technical and creative challenges:

- Ball Physics: Creating realistic ball movement with variable bounce angles was challenging. I addressed this by implementing the BallBounceAngle function, incorporating random jitter and a minimum bounce angle using mathematical calculations.
- AI Response: Designing a computer opponent that was challenging but playable required precise calibration of response times and movement speed. I used a delay variable (aiDelayTimer) and variable speed based on difficulty level.
- Keyboard Input Handling: Detecting key presses and releases in real-time was challenging. I used a keys array and a lastKeyPressedTime variable to prevent unwanted paddle movement.
- DLL Usage: Organizing game logic in a DLL required proper management of API interfaces and callback functions. I defined functions like SetGameCallbacks in the DLL to enable communication between the main code and the DLL.